

Perigren, Jomar A.

BSIT A706

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *
import math

# Diamond vertices (octahedron shape)
vertices = [
    [0, 1, 0],    # 0: Top vertex
    [1, 0, 0],    # 1: Right
    [0, 0, 1],    # 2: Front
    [-1, 0, 0],   # 3: Left
    [0, 0, -1],   # 4: Back
    [0, -1, 0]    # 5: Bottom vertex
]

# Define triangles using GL_TRIANGLES
triangles = [
    # Top pyramid (4 triangular faces)
    [0, 1, 2], # Top-Right-Front
    [0, 2, 3], # Top-Front-Left
    [0, 3, 4], # Top-Left-Back
    [0, 4, 1], # Top-Back-Right

    # Bottom pyramid (4 triangular faces)
    [5, 2, 1], # Bottom-Front-Right
    [5, 3, 2], # Bottom-Left-Front
    [5, 4, 3], # Bottom-Back-Left
    [5, 1, 4]  # Bottom-Right-Back
]

# Colors for each face (RGB)
colors = [
    [1, 0, 0],    # Red
    [0, 1, 0],    # Green
    [0, 0, 1],    # Blue
    [1, 1, 0],    # Yellow
    [1, 0, 1],    # Magenta
    [0, 1, 1],    # Cyan
    [1, 0.5, 0],  # Orange
    [0.5, 0, 1]   # Purple
]

def draw_diamond():
```

Perigren, Jomar A.

BSIT A706

```
glBegin(GL_TRIANGLES)
for i, triangle in enumerate(triangles):
    glColor3fv(colors[i])
    for vertex_idx in triangle:
        glVertex3fv(vertices[vertex_idx])
glEnd()
```

```
def main():
    pygame.init()
    display = (800, 600)
    pygame.display.set_mode(display, DOUBLEBUF | OPENGL)
    pygame.display.set_caption("3D Diamond - Use Arrow Keys, W/S, A/D, Q/E")

    # Set up perspective
    gluPerspective(45, (display[0] / display[1]), 0.1, 50.0)
    glTranslatef(0.0, 0.0, -5)

    # Enable depth testing
    glEnable(GL_DEPTH_TEST)

    # Transformation variables
    rotation_x = 0
    rotation_y = 0
    rotation_z = 0
    scale = 1.0
    translate_x = 0
    translate_y = 0
    translate_z = 0

    clock = pygame.time.Clock()

    print("\n=== 3D Diamond Controls ===")
    print("ROTATION:")
    print(" Arrow Up/Down: Rotate around X-axis")
    print(" Arrow Left/Right: Rotate around Y-axis")
    print(" Q/E: Rotate around Z-axis")
    print("\nSCALE:")
    print(" W: Scale up")
    print(" S: Scale down")
    print("\nTRANSLATION:")
    print(" A/D: Move left/right")
    print(" Z/X: Move up/down")
    print(" C/V: Move forward/backward")
    print("\nR: Reset transformations")
```

Perigren, Jomar A.

BSIT A706

```
print("ESC: Exit")
print("=====\n")

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                running = False
            elif event.key == pygame.K_r:
                # Reset transformations
                rotation_x = rotation_y = rotation_z = 0
                scale = 1.0
                translate_x = translate_y = translate_z = 0

# Keyboard controls
keys = pygame.key.get_pressed()

# Rotation controls
if keys[pygame.K_UP]:
    rotation_x += 2
if keys[pygame.K_DOWN]:
    rotation_x -= 2
if keys[pygame.K_LEFT]:
    rotation_y -= 2
if keys[pygame.K_RIGHT]:
    rotation_y += 2
if keys[pygame.K_q]:
    rotation_z -= 2
if keys[pygame.K_e]:
    rotation_z += 2

# Scale controls
if keys[pygame.K_w]:
    scale += 0.01
if keys[pygame.K_s]:
    scale = max(0.1, scale - 0.01)

# Translation controls
if keys[pygame.K_a]:
    translate_x -= 0.02
if keys[pygame.K_d]:
```

Perigren, Jomar A.

BSIT A706

```
        translate_x += 0.02
    if keys[pygame.K_z]:
        translate_y += 0.02
    if keys[pygame.K_x]:
        translate_y -= 0.02
    if keys[pygame.K_c]:
        translate_z += 0.02
    if keys[pygame.K_v]:
        translate_z -= 0.02
```

```
# Clear screen and depth buffer
```

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

```
# Save current matrix
```

```
glPushMatrix()
```

```
# Apply transformations in order: translate -> rotate -> scale
```

```
glTranslatef(translate_x, translate_y, translate_z)
```

```
glRotatef(rotation_x, 1, 0, 0)
```

```
glRotatef(rotation_y, 0, 1, 0)
```

```
glRotatef(rotation_z, 0, 0, 1)
```

```
glScalef(scale, scale, scale)
```

```
# Draw the diamond
```

```
draw_diamond()
```

```
# Restore matrix
```

```
glPopMatrix()
```

```
pygame.display.flip()
```

```
clock.tick(60)
```

```
pygame.quit()
```

```
if __name__ == "__main__":
```

```
    main()
```